

Design and Implementation of a ROS 2 Pick-and-Place Color Sorting System Using OpenCV and the Franka Panda Robot Arm

Presented by

- Abdessamad Britah
- Sarah Hamdane
- Yasmine Djellal
- Amina Yahiaoui
- Meryem MERHOUNI



ROS 2 Workspace Architecture

What is colcon build?

colcon

Collective Construction — standard ROS 2 meta-build tool

Discovers

All packages in src/ automatically via package.xml

Builds

Each package into install/ with layered overlays

colcon build

→ **build/**

→ **install/**

panda_ws/ Project Workspace

```

📁 src/           # all source packages
  📁 panda_description/  # URDF, meshes
  📁 panda_controller/   # joint & gripper
  📁 panda_moveit/       # MoveIt 2 config
  📁 panda_vision/       # OpenCV detection
  📁 panda_bringup/      # launch files
📁 build/         # compilation artifacts
📁 install/       # ← source this!
📄 log/          # build & runtime logs
```

Each subdirectory of src/ is a self-contained ROS 2 package with its own build system, dependencies, and node definitions.

Sourcing install/setup.bash
activates all built packages
for the current shell session.

Package Structure & ROS 2 Communication

Anatomy of a ROS 2 Package

package.xml

Declares name, version, and build/exec dependencies

CMakeLists.txt

Build rules for C++ (ament_cmake)

setup.py

Build rules for Python (ament_python)

launch/*.py

Orchestrates multiple nodes simultaneously

```
$ colcon build --symlink-install
# builds all packages in src/ and
# symlinks Python files for editing
```

DDS Middleware — How Nodes Talk

panda_vision
(Color Detection)

panda_controller
(Joint Control)

DDS Bus

panda_moveit
(Motion Planning)

Gazebo Simulation
(Physics Engine)

Key ROS 2 Topics:

- ▶ /detected_objects
- ▶ /joint_states
- ▶ /move_group/goal
- ▶ /gazebo/model_states

Robot Modeling — URDF Links & Physical Parameters

URDF — an XML file describing a robot: links, joints, visual meshes, collision geometry, and dynamic parameters. Used by Gazebo, MoveIt 2, RViz.

A URDF Link — 3 Sub-elements

```
<link name="panda_link1">
  <inertial>
    <mass value="4.97"/>
    <inertia ixx="0.703" ixy="-0.000139"
              ixz="0.006772" iyy="0.706"
              iyz="0.019" izz="0.009"/>
  </inertial>
  <visual>
    <geometry>
      <mesh filename="panda_link1.dae"/>
    </geometry>
    <material name="white"/>
  </visual>
  <collision>
    <geometry>
      <mesh filename="panda_link1_col.stl"/>
    </geometry>
  </collision>
</link>
```

Franka Panda — 12 Links

Link	Role	Mass
panda_link0	Base (fixed)	—
panda_link1	Shoulder rotation	4.97 kg
panda_link2	Shoulder tilt	0.65 kg
panda_link3	Upper arm	3.23 kg
panda_link4	Elbow	3.59 kg
panda_link5	Forearm	1.22 kg
panda_link6	Wrist rotation	1.66 kg
panda_link7	Wrist tilt	0.735 kg
panda_hand	Gripper palm	0.73 kg

Inertia tensor I defines resistance to rotation — critical for accurate Gazebo dynamics.

URDF Joints & Denavit-Hartenberg Parameters

Joint Types in URDF

revolute

Rotation with limits — all 7 Panda joints

prismatic

Linear sliding motion — gripper fingers

fixed

No movement — structural connection

```
<joint name="panda_joint1" type="revolute">
  <parent link="panda_link0"/>
  <child link="panda_link1"/>
  <origin xyz="0 0 0.333" rpy="0 0 0"/>
  <axis xyz="0 0 1"/> <!-- Z-axis rotation -->
  <limit lower="-2.897" upper="2.897"
    effort="87" velocity="2.175"/>
</joint>
```

Denavit-Hartenberg Parameters — Franka Panda (7 DOF)

DH convention: d (link offset) · θ (joint angle) · a (link length) · α (twist angle)

Joint	d (m)	a (m)	α (rad)	θ range
1	0.333	0	0	$-166^\circ..166^\circ$
2	0	0	$-\pi/2$	$-101^\circ..101^\circ$
3	0.316	0	$+\pi/2$	$-166^\circ..166^\circ$
4	0	+0.0825	$+\pi/2$	$-176^\circ..-4^\circ$
5	0.384	-0.0825	$-\pi/2$	$-166^\circ..166^\circ$
6	0	0	$+\pi/2$	$-1^\circ..215^\circ$
7	0.107	+0.088	$+\pi/2$	$-166^\circ..166^\circ$

Transformation Matrix (per joint):

$${}^{i-1}T_i = \text{Rot}_z(\theta_i) \cdot \text{Trans}_z(d_i) \cdot \text{Trans}_x(a_i) \cdot \text{Rot}_x(\alpha_i)$$

$$\text{End-effector: } T_{07} = T_{01} \cdot T_{12} \cdot T_{23} \cdot T_{34} \cdot T_{45} \cdot T_{56} \cdot T_{67}$$

Simulation in Gazebo — Physics Engine & Real-World Behavior

Gazebo is a 3D physics-based robot simulator integrated with ROS 2, providing realistic gravity, inertia, friction, and contact forces before deploying on hardware.

↓ Gravity & Dynamics

Applies $g = 9.81 \text{ m/s}^2$ to all links. The inertia tensor I from the URDF determines how each link responds to torques.

★ Collision Detection

Uses simplified collision meshes to detect contact. ODE resolves penetrations and computes contact forces at each time-step.

🔒 Friction Models

Gazebo applies Coulomb friction: $F_{\text{friction}} \leq \mu \cdot F_{\text{normal}}$. Critical for the color-sorting pick task.

⚙️ Joint Controllers

ros2_control PID plugins drive each joint: $\tau = K_p \cdot e + K_i \cdot \int e + K_d \cdot \dot{e}$, where e is the angle error.

URDF → Gazebo Pipeline:

panda.urdf.xacro



xacro parser



robot_description



Gazebo spawner



Physics sim

ROS 2 Controllers Setup

ros2_control translates Movelt 2 trajectories into joint torques via PID feedback

JointTrajectoryController

- Receives trajectories from Movelt 2
- Interpolates waypoints in time
- Sends commands to robot joints
- PID loop for trajectory tracking

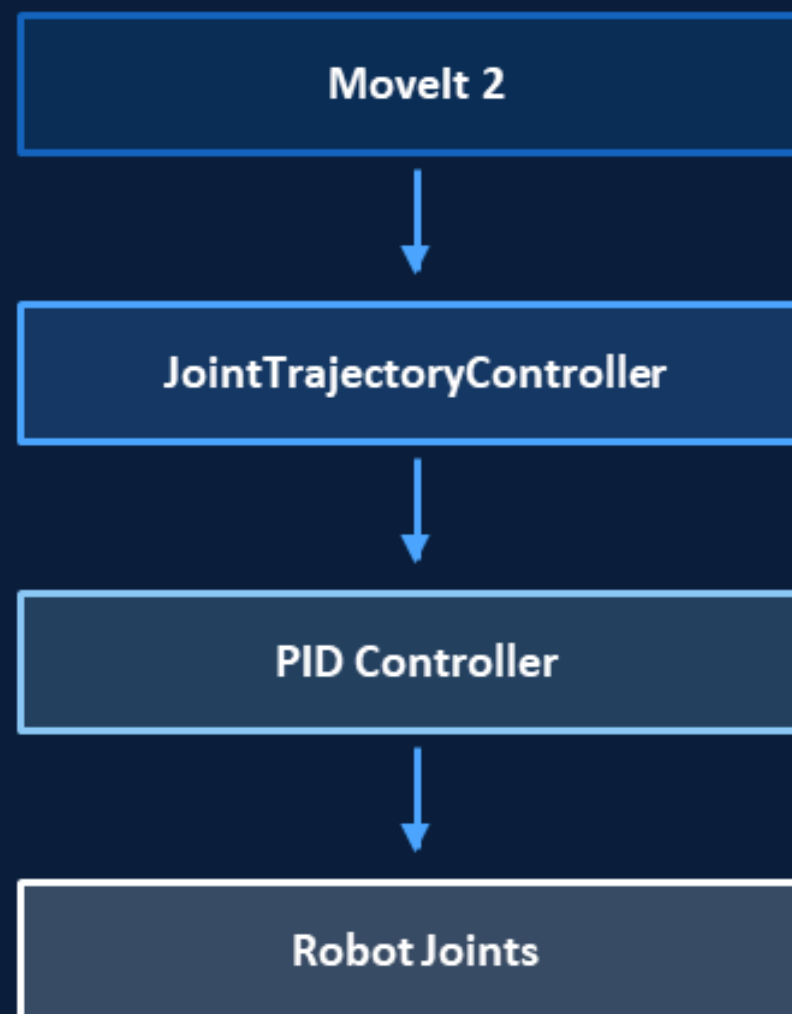
GripperActionController

- Controls gripper open / close
- Supports grasp action interface
- Configurable effort & speed

JointStateBroadcaster

- Publishes encoder feedback
- Sends /joint_states topic at 50 Hz
- Used by RViz & Movelt 2

EXECUTION FLOW



PID CONTROL LAW

$$\tau = K_p \cdot e + K_i \int e \, dt + K_d \cdot \dot{e}$$

PID compensates trajectory tracking errors for accurate joint motion

panda_controllers.yaml

```
panda_arm_controller:
  type: joint_trajectory_controller

joints:
  - joint1  - joint2  - joint3

pid:
  joint1: {p: 250, d: 5}
  joint2: {p: 300, d: 8}
```

8

Controller Configuration & Python Node

YAML config + Python orchestration bridge MoveIt 2 to the hardware layer

`pick_and_place.launch.py`

Launches all ROS 2 nodes in parallel

- **gazebo_ros**
Physics simulation
- **robot_state_publisher**
TF tree from URDF
- **move_group**
MoveIt 2 planner
- **ros2_control_node**
Hardware interface
- **panda_vision**
OpenCV color detect
- **rviz2**
Visualization

`pick_and_place.py`

High-level task coordination node

```
# Subscribe to detected color
color_sub = create_subscription(...)

# Plan motion with MoveIt 2
arm.set_pose_target(pick_pose)
arm.go(wait=True)

# Gripper control
gripper.close()

# Place to target bin
arm.set_named_target('bin_R')
arm.go(wait=True)
```

Key ROS 2 Topics

Asynchronous distributed communication

`/detected_color`

Vision → Python node

`/joint_states`

Joint encoder feedback

`/move_group/goal`

MoveIt 2 motion goals

`/gazebo/model_states`

Simulation object states

Vision



Python Node



MoveIt 2



ros2_control



Robot

7 DOF

Panda robot arm
joint chain depth

4x4 Matrix

Homogeneous transform
combining rotation & translation

Denavit-Hartenberg Convention

- d** — link offset (along z)
- θ** — joint angle (around z)
- a** — link length (along x)
- α** — twist angle (around x)



Robot Kinematics

Mathematical foundations for motion planning with MoveIt 2

Forward Kinematics (FK)

joint angles → **end-effector position**

Calculates position & orientation of the end-effector from the 7 known joint angles of the Panda arm.

$$T_{07} = T_{01} \cdot T_{12} \cdot T_{23} \dots T_{67}$$

Inverse Kinematics (IK)

target position → **joint angles**

Inverse problem: computes the 7 joint angles needed to reach a given Cartesian pose. Solved by MoveIt 2 via the IK solver (KDL).

Rotation Matrices & Transforms

Each link i applies a 4x4 homogeneous transform combining rotation and translation:

$${}^{i-1}T_i = \text{Rot_z}(\theta_i) \cdot \text{Trans_z}(d_i) \cdot \text{Trans_x}(a_i) \cdot \text{Rot_x}(\alpha_i)$$

DH Parameters (Denavit-Hartenberg):

- d** — offset along z-axis (link offset)
- θ** — rotation about z (joint angle)
- a** — length along x (link length)
- α** — twist about x (twist angle)

7 DOF → 7 chained transforms from base to end-effector
(panda_link0 → panda_hand)

Intelligent Automation

What is MoveIt 2?

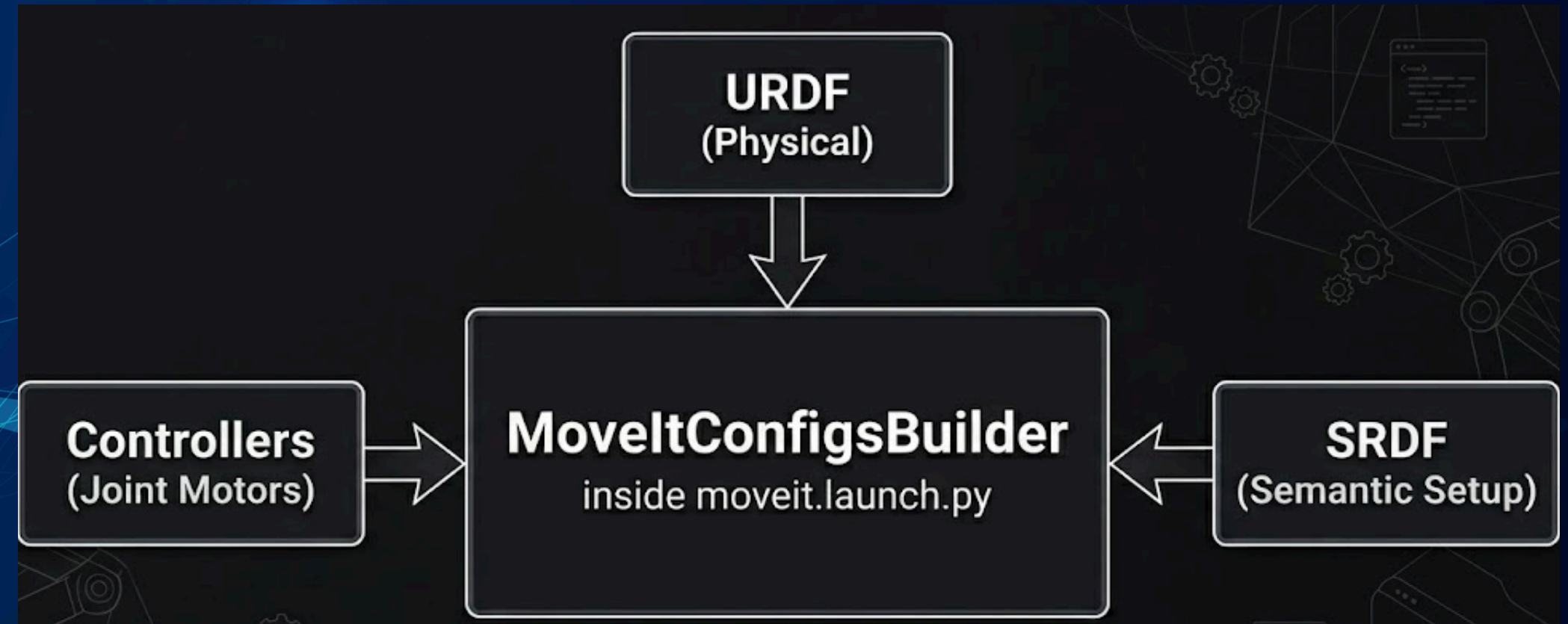
The central manipulation "brain" that controls how the arm moves.

moveit.launch.py:

Orchestrates the system by initializing the central `move_group` node and loading parameter runtimes

Programmatic Integration:

Utilizes `MoveItConfigsBuilder` to dynamically aggregate and inject configuration files into the ROS 2 ecosystem.



**URDF**

Physical Properties

**SRDF**

Semantic Setup

**Controllers YAML**

Hardware Map

**MoveItConfigsBuilder**

inside moveit.launch.py

- Loads & binds all config files
- Initializes node parameters
- Injects into ROS 2 ecosystem

**move_group
Node**

Central Orchestrator



Inverse Kinematics

IK Solver

Translates 3D Cartesian coordinates (x, y, z) into precise joint angles for the 7-DOF arm.



Motion Planning

Path Finding

Traces a continuous, collision-free pathway through configuration space from state A $\rightarrow$ B.



Collision Avoidance

Safety Guard

Computes real-time distance matrices against environment geometry and sorting bins.



Trajectory Parameterization

Smooth Profiles

Applies velocity & acceleration constraints to minimize mechanical wear and stress.

Movelt 2 — Simulation Demo Pipeline



Demo Workflow: Boot stack → System confirms **Ready to plan** → Run node with **target_color:=R** → Autonomous pick-and-place loop

Camera setup in the simulation

The robot's eye — a fixed overhead camera streaming live images to ROS 2

What the camera is

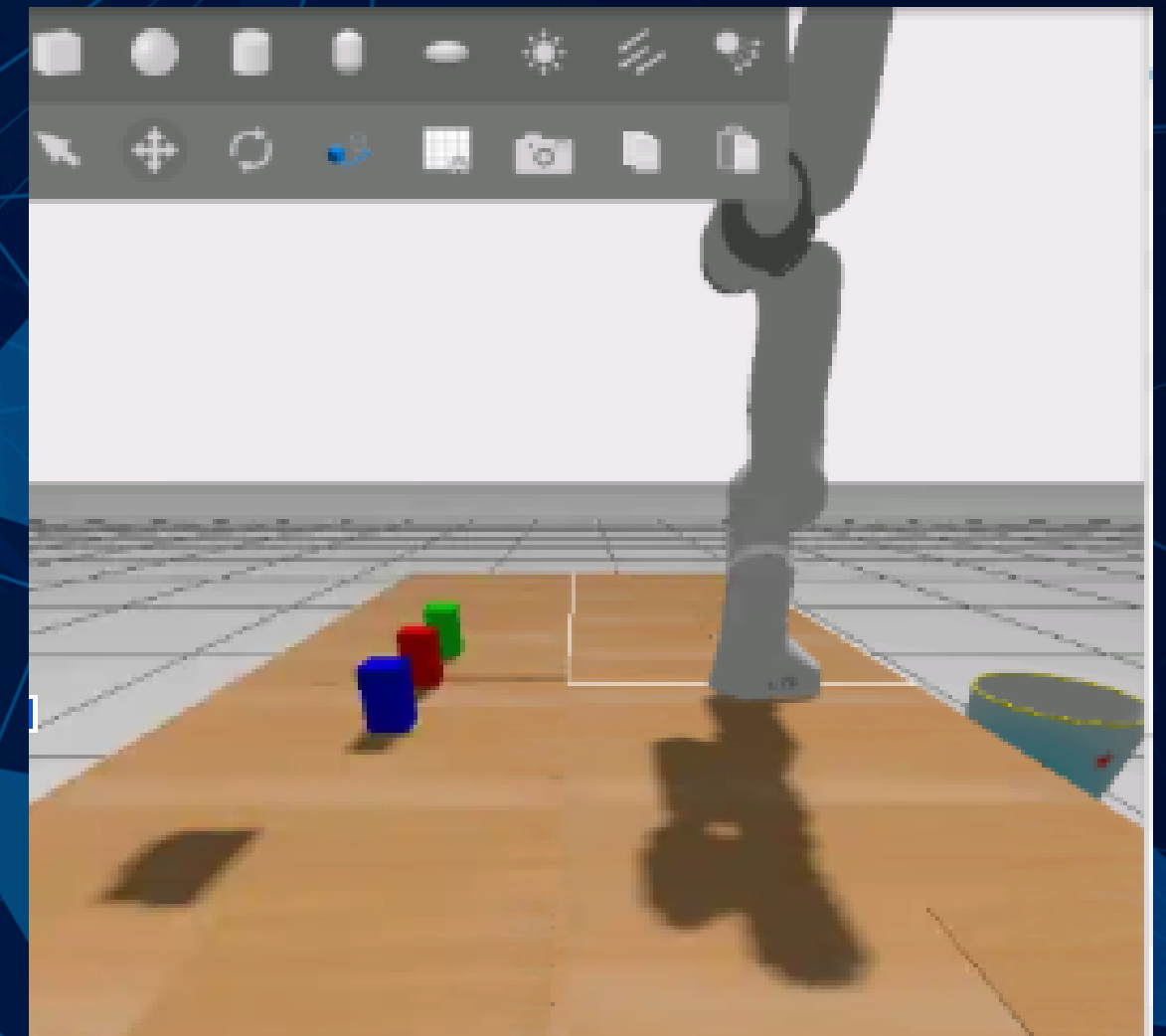
A fixed overhead camera — not mounted on the arm. It sits 1 meter above the table looking straight down at the workspace. This keeps the system simple: the camera never moves

How it connects to ROS 2

A Gazebo camera plugin makes it stream live images every frame to the topic `/camera/image_raw`. The color detector node subscribes to this topic and processes each frame.

Intrinsic parameters — used for 3D projection later

These values come from the camera model in the SDF file. They describe the camera's lens geometry to convert pixels into real-world meters.



585

f_x — focal length X

588

f_y — focal length Y

320

c_x — image center X

160

c_y — image center Y

From RGB to HSV — color space

Why we convert colors before detecting — the key to robust detection

RGB — problem

Color and brightness are mixed together. A red box in shadow looks completely different from red in bright light. Thresholding fails.

VS

HSV — solution

Hue = color identity, separate from brightness. A red box stays "red" no matter the lighting. We only threshold the Hue channel.

H

Hue
color identity
0° → 360°

S

Saturation
color purity
0 → 255

V

Value
brightness
0 → 255

HSV thresholds used in `color_detector.py`

Red

Hue: 0 – 10
Sat: 120 – 255
Val: 70 – 255

Green

Hue: 55 – 60
Sat: 200 –
255
Val: 200 – 255

Blue

Hue: 90 – 128
Sat: 200 – 255
Val: 200 – 255

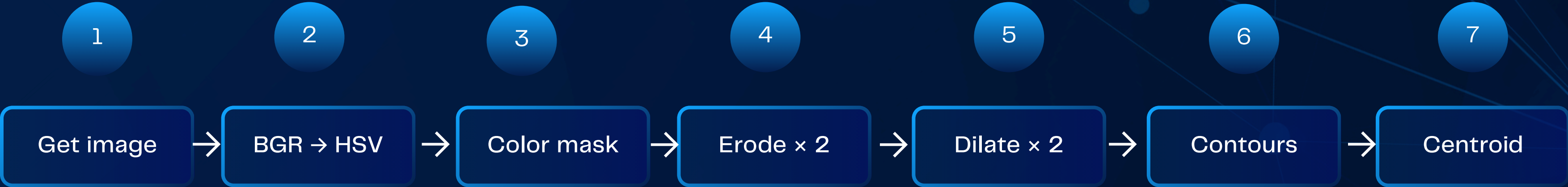
How a mask is created

For each color, `cv2.inRange(hsv, lower, upper)` produces a binary image — white where the color is detected, black everywhere else.

`cv2.cvtColor(frame, COLOR_BGR2HSV)`
one line converts the whole image

Color detection pipeline

7 steps – from raw camera image to the exact pixel position of each colored box



Step 1–2

Subscribe to camera topic, receive image, convert immediately to HSV color space.

Step 3 – mask

Binary image: white = color detected, black = everything else.
`cv2.inRange(hsv, lower, upper)`

Step 4–5 noise removal

Erode removes tiny false dots. Dilate restores the real box shape. Only the actual box survives.
 erode → dilate (×2 each)

Step 6–7 locate box

Find contours on the clean mask. Draw bounding rectangle. Compute center pixel.
 $cx = x + w//2$
 $cy = y + h//2$

Output of this pipeline: pixel coordinates (cx_pix, cy_pix) of each colored box – ready for 3D projection

Pixel to 3D coordinates — projection

The pinhole camera model: converting a pixel position into a real-world 3D point

The problem

After the pipeline we know where the box is in the image (pixel). But the robot needs to know where it is in the real world (meters). We need to reverse the camera projection.

Pinhole camera model — formulas

$$Z = 0.1 \quad (\text{camera height — assumed})$$

$$Y = (cx_pix - cx) \times Z / fx \times (-10)$$

$$X = (cy_pix - cy) \times Z / fy$$

WHAT EACH VARIABLE MEANS

cx_pix, cy_pix — center pixel of the detected box

cx, cy — image center (principal point of the lens)

fx, fy — focal lengths: how pixels relate to meters

Z — depth (we know the camera is 1 m above the table)

camera (fixed above table)

Camera

Z = 0.1 (depth known)

pixel (cx_pix, cy_pix)

pixel in image

formula applied ↓

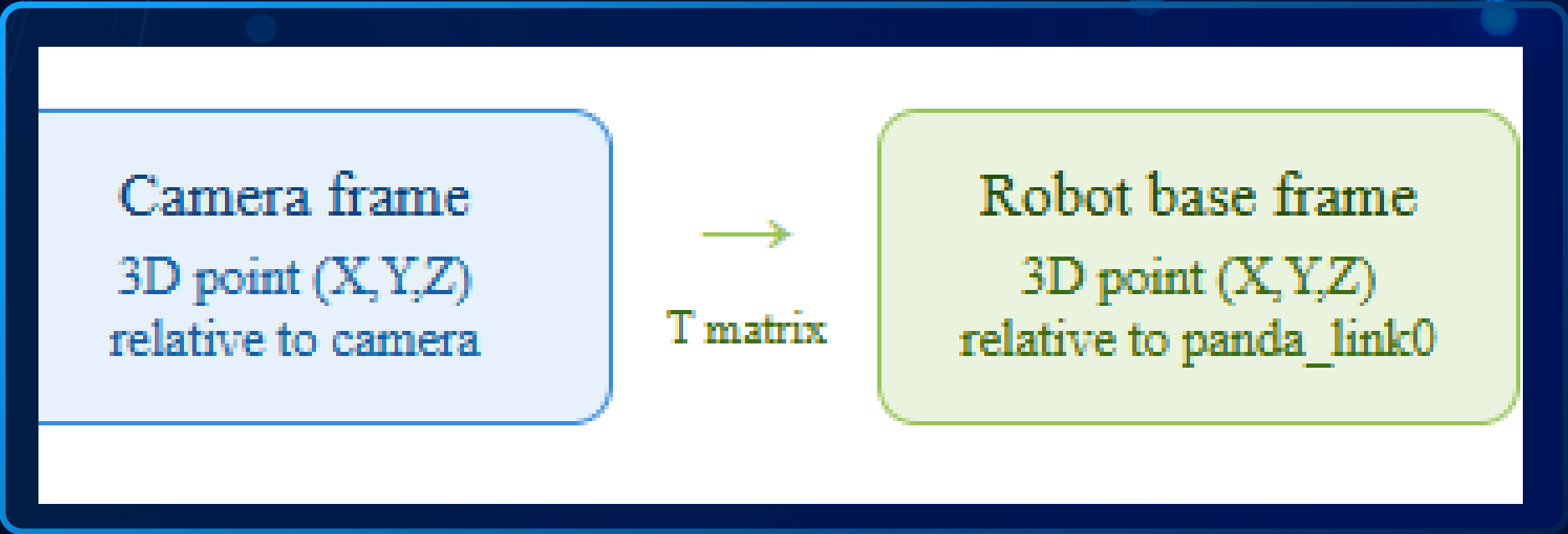
3D point (X, Y, Z)

in camera frame

● depth Z ● pixel coords ● 3D result

TF2 coordinate transforms

Moving from camera frame to robot base frame using a 4x4 homogeneous matrix



4x4 Homogeneous Transformation Matrix T

$$\begin{bmatrix} r_{00} & r_{01} & r_{02} & | & t_x \\ r_{10} & r_{11} & r_{12} & | & t_y \\ r_{20} & r_{21} & r_{22} & | & t_z \\ 0 & 0 & 0 & | & 1 \end{bmatrix}$$

R (3x3) — rotation from quaternion

t (3x1) — translation tx, ty, tz

- TF2 looks up the transform: `camera_link → panda_link0` → returns translation + quaternion
- Quaternion is converted to a 3x3 rotation matrix R using `quaternion_matrix()`
- We build the full 4x4 matrix T and apply: `pt_base = T @ [X, Y, Z, 1.0]`

THE OPERATION — ONE MATRIX MULTIPLY

`pt_cam = [X, Y, Z, 1.0]`

`pt_base = T @ pt_cam` ← robot coordinates

Output: (X, Y, Z) in panda_link0 frame — directly usable by MoveIt to move the arm

ROS 2 integration & result

The color detector node – connecting vision to the robot arm with live proof

/camera/image_raw → ColorDetector node → /color_coordinates → pick_and_place.py

What the node does – all in one loop

On every camera frame: convert to HSV → detect colors → clean mask → find contours → compute pixel centroid → project to 3D → apply TF2 transform → publish coordinates. All 7 steps from the previous slides run continuously in `rclpy.spin(node)`.

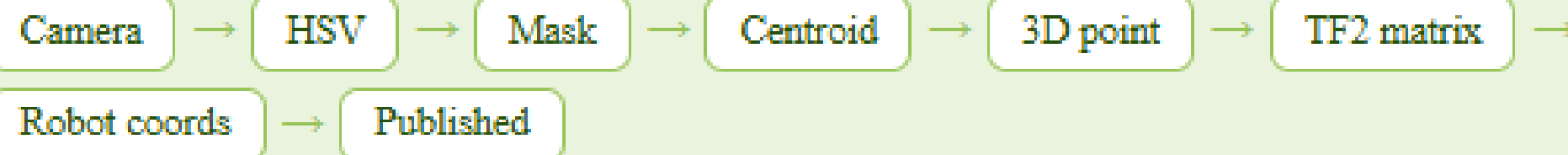
`cv_bridge` converts ROS image to OpenCV format

`tf2_ros` looks up coordinate frame transforms

`opencv` runs the full vision pipeline

`rclpy` ROS 2 Python node API

Full vision pipeline in one view



Live terminal output – real result from our demo

```

#1 #2 #3 #4
[INFO] [1778844795.076918483] [color_detector]: B,0.600,0.206,1.100
[INFO] [1778844795.573823748] [color_detector]: R,0.600,0.000,1.100
[INFO] [1778844795.648731664] [color_detector]: G,0.600,-0.206,1.100
[INFO] [1778844795.651519117] [color_detector]: B,0.600,0.206,1.100
[INFO] [1778844795.980657568] [color_detector]: R,0.599,-0.084,1.100
[INFO] [1778844795.982009688] [color_detector]: G,0.600,-0.209,1.100
[INFO] [1778844796.049821465] [color_detector]: B,0.600,0.206,1.100
[INFO] [1778844796.669026177] [color_detector]: R,0.600,0.000,1.100
[INFO] [1778844796.670209921] [color_detector]: G,0.600,-0.206,1.100
[INFO] [1778844796.671140171] [color_detector]: B,0.600,0.206,1.100
[INFO] [1778844796.969126715] [color_detector]: R,0.599,-0.084,1.100
[INFO] [1778844796.973793637] [color_detector]: G,0.600,-0.209,1.100
[INFO] [1778844796.976558423] [color_detector]: B,0.600,0.206,1.100
[INFO] [1778844797.768960365] [color_detector]: R,0.600,-0.084,1.100
[INFO] [1778844797.770456791] [color_detector]: G,0.600,-0.209,1.100
[INFO] [1778844797.849035387] [color_detector]: B,0.600,0.206,1.100
[INFO] [1778844797.875442505] [color_detector]: R,0.600,0.000,1.100
[INFO] [1778844797.876542780] [color_detector]: G,0.600,-0.206,1.100
[INFO] [1778844797.877492661] [color_detector]: B,0.600,0.206,1.100
  
```

What the robot does with this

`pick_and_place.py` subscribes to `/color_coordinates` and waits for the target color. When it receives R, 0.600, 0.000, 1.100 – it sends those exact coordinates to MoveIt, which moves the arm to pick the red box. Vision and motion are fully decoupled through ROS 2 topics.